Replication & Quorums

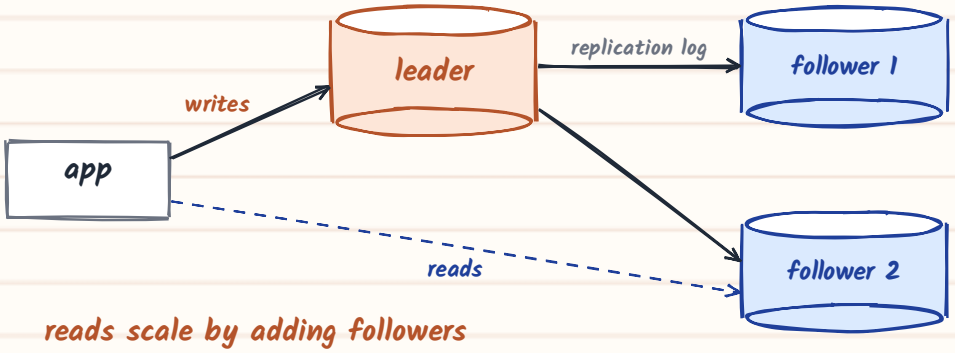
Lesson 12 · leader-follower, multi-leader & leaderless

In this lesson → ① Why replicate ② Leader-follower ③ Sync vs async ④ Replication lag ⑤ Failover ⑥ Multi-leader ⑦ Leaderless ⑧ Compare ⑨ Interview Q&A

Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

> **Replication** = keeping copies of the same data on several machines, for **availability**, **read scale** and **lower latency**.

1 Leader-follower (primary-replica)



reads scale by adding followers

→ All writes → leader
→ Leader streams its log → followers
→ Reads from any follower → add followers to scale reads

2 Synchronous vs asynchronous

	Synchronous	Asynchronous
Leader waits?	yes	no
Write latency	higher	low
Data loss on failover	none	recent writes may be lost
Typical	semi-sync: wait for 1 follower	default

3 Replication lag

✗ Post a photo → refresh → gone (read hit a stale follower)

✓ **Read-your-writes**: own recent data from the leader

✓ **Monotonic reads**: same user → same replica

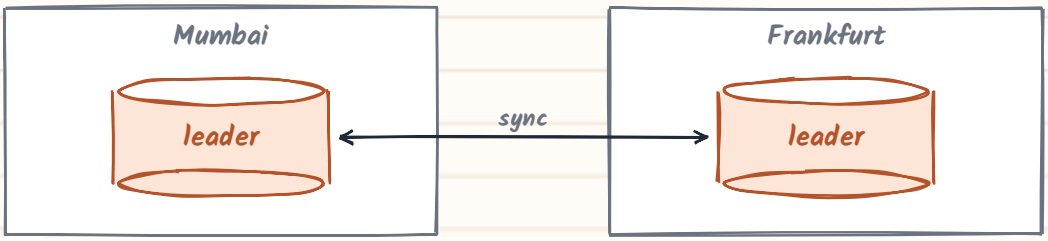
✓ Track versions / LSN and wait until caught up

4 Failover

→ Detect (missed heartbeats) → promote most up-to-date follower → reroute clients

→ Risks: lost async writes, **split brain** → use fencing tokens

5 Multi-leader



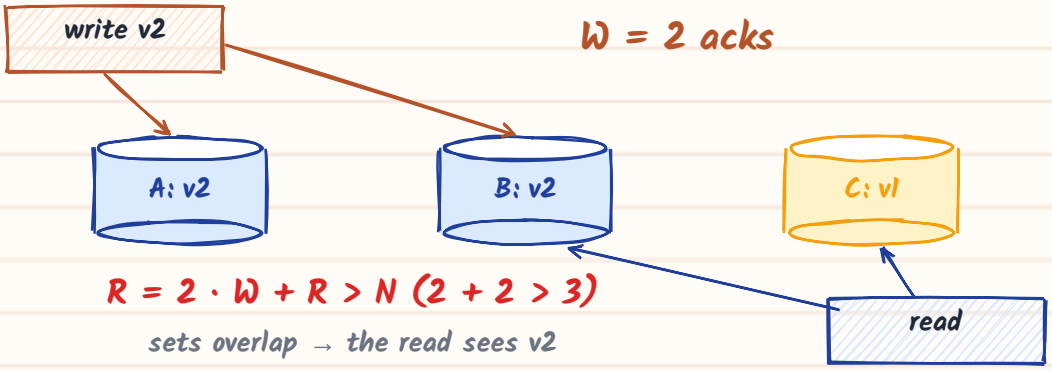
same caption edited in both → conflict!

resolve: last-write-wins · merge (CRDT) · ask the user

→ Leader per region, local writes, async sync between leaders

→ Conflicts: last-write-wins (loses data), merge / CRDTs, ask user

6 Leaderless + quorums



✓ Write to N, wait for **W** acks; read from **R**, take newest

✓ **W + R > N** → read and write sets overlap

✓ Common: N=3, W=2, R=2

✓ **Read repair** + **hinted handoff** fix stale copies

	Leader-follower	Multi-leader	Leaderless
Writes	one leader	leader per region	any W replicas
Conflicts	none	yes	yes (versions)
Used by	Postgres, MySQL	multi-region apps	Cassandra, DynamoDB

8 Interview Corner

Q. Why replicate?

A. Availability when nodes fail, read scaling, and lower latency near users.

Q. Sync or async replication?

A. Sync: no loss but slower. Async: fast but may lose recent writes. Semi-sync is common.

Q. What is replication lag and one fix?

A. Followers are behind the leader. Read-your-writes: read own recent data from the leader.

Q. What is split brain?

A. Two nodes both think they are leader. Prevent with fencing tokens / consensus.

Q. What does W + R > N give you?

A. Every read overlaps at least one replica with the latest write.

⚡ Revise in 60 seconds

- → **Leader-follower**: default
- → Sync = safe, async = fast
- → **Lag** → read-your-writes
- → Failover + fencing
- → **Multi-leader**: regions
- → Conflicts: LWW / CRDT
- → **Leaderless**: quorums
- → **W + R > N**

Next lesson → SD-13
Sharding and consistent hashing.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**